

TABLE OF CONTENTS

How to Build a Robust Flutter App Initialization Flow with Riverpod	1
Handling App Startup Errors: The Basics	2
The Limitation	2
Improved Error Handling with a StatefulWidget	3
Implementation	4
Next Steps	5
Asynchronous Dependency Initialization with Riverpod	5
Eager Provider Initialization with a Child Widget	6
How It Works	8
Accessing Eagerly-Initialized Providers with requireValue	8
Important Note About URL Navigation and Deep Links	9
The Solution: Introducing a RootAppWidget	9
How It Works	10
Real-World Example	11
Summary: Stateful App Initialization with Router	11
Common Questions	12
How to Eagerly Initialize Multiple Providers?	13
What About Programmer or Configuration Errors?	13
How to Implement the Retry Logic?	14
Can Any Provider be Eagerly Initialized?	15
How to Transition Between the Splash, Loading, and Main UI Screens?	16
Can I Use Provider Overrides as a Simpler Alternative for Asynchronous Initialization?	16
Conclusion	17
Real-World Example	18
What's Next?	18
New Course: Flutter in Production	18
Flutter In Production	19
Want More?	19
Flutter Foundations Course	19
Flutter & Firebase Masterclass	19
The Complete Dart Developer Guide	20
Flutter Animations Masterclass	20



How to Build a Robust Flutter App Initialization Flow with Riverpod

#dart

#flutter

#state-management

#riverpod



Andrea Bizzotto

Updated Dec 2, 2024 · 15 min read



Source code on GitHub

First impressions matter—especially when users launch your Flutter app for the first time. A glitchy startup or a poor onboarding experience can frustrate users, leading to bad reviews or even uninstalls.

So, how do you ensure your app's startup code is rock-solid, handles errors gracefully, and scales to support features like deep links or URL-based navigation?

In this article, we'll start simple and build towards a robust, production-ready solution. Here's the game plan:

1. **Handling App Startup Errors in `main`**: The basic (and naive) approach.
2. **Improved Error Handling with a `StatefulWidget`**: Adding loading UI and retry functionality.
3. **Asynchronous Dependency Initialization with Riverpod**: A more scalable solution.

We'll start by implementing a `StatefulWidget` to display a loading UI while initializing the app. You'll also learn how to handle errors and allow users to retry if something goes wrong.

Next, we'll explore **eager provider initialization** with Riverpod. This technique lets you preload dependencies asynchronously and access them synchronously later using `requireValue`.

Finally, we'll tweak the setup to ensure the flow supports **URL navigation and deep links** without breaking.

By the end, you'll have a robust app initialization strategy that's ready for production. 🚀

Ready to dive in? Let's go! 📌

Handling App Startup Errors: The Basics

Let's start with a simple example:

```
void main() async {  
  await someAsyncCodeThatMayThrow();  
  runApp(const MaterialApp(home: MainApp()));  
}
```

What happens if the code above throws an exception?

The app will **fail to call** `runApp` and end up stuck on the splash screen—leaving your users confused and frustrated. Ouch! 🤖

To improve this slightly, we can wrap the code in a `try-catch` block:

```
void main() async {  
  try {  
    await someAsyncCodeThatMayThrow();  
    runApp(const MaterialApp(home: MainApp()));  
  } catch (e, st) {  
    // TODO: register the global error handlers:  
https://docs.flutter.dev/testing/errors  
    log(e.toString(), stackTrace: st);  
    runApp(const MaterialApp(home: AppStartupErrorWidget(e)));  
  }  
}
```

Now, if something goes wrong, we log the error and show an `AppStartupErrorWidget` with an appropriate error message. This provides some feedback to the user instead of just hanging on the splash screen.

The Limitation

While this approach is better than nothing, it's still far from ideal. Why? Because it doesn't allow the user to **retry**. If the startup process fails, the only option is to close and restart the app—a frustrating experience.

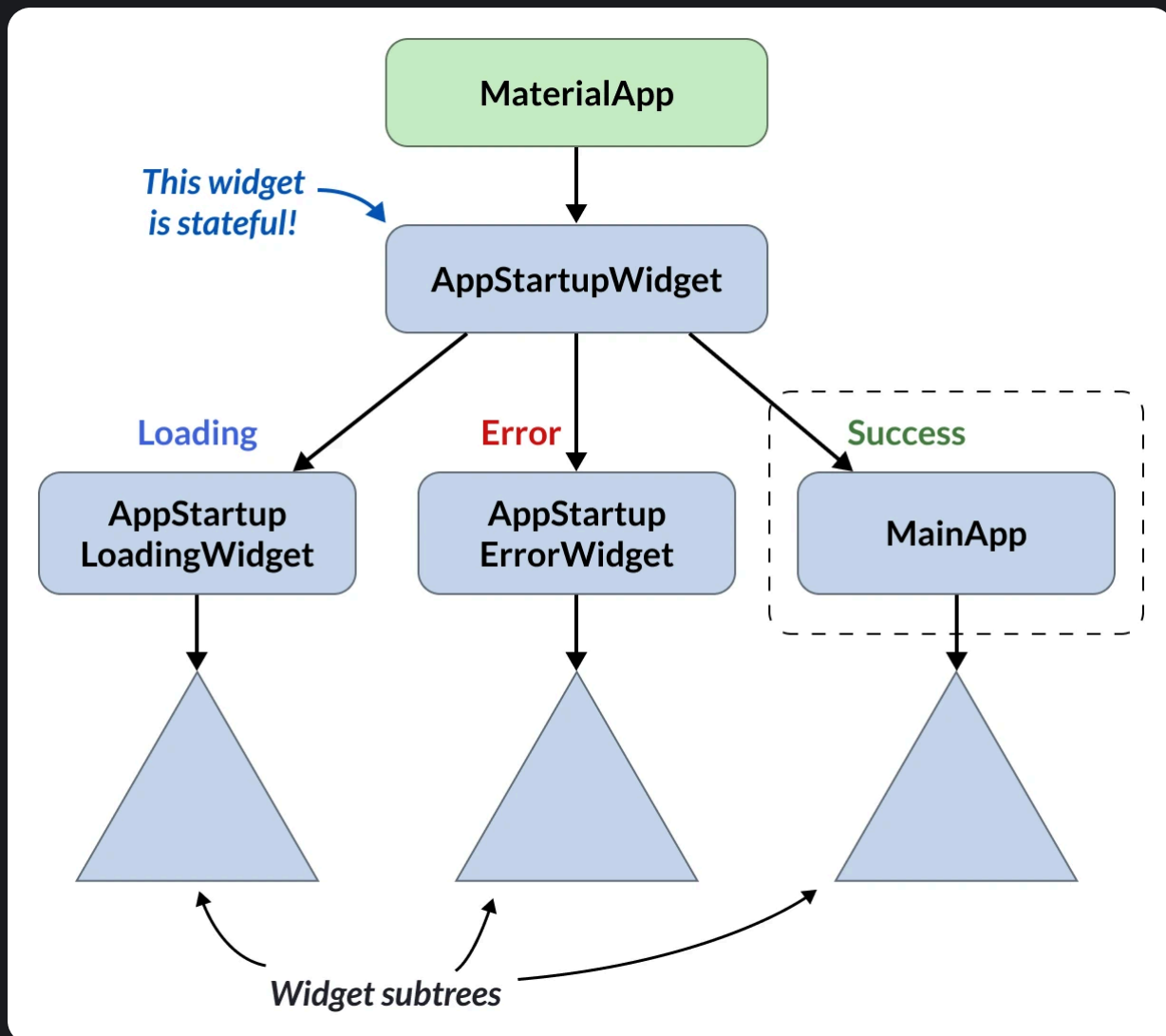
Let's see how we can improve on this!

Improved Error Handling with a StatefulWidget

To enhance the user experience, let's tackle a few key requirements for app startup:

- Display a **loading screen** during initialization.
- If initialization fails, show an **error message** with a “Retry” button.
- If initialization succeeds, show the **main app UI**.

We can handle these scenarios—**loading**, **error**, and **success**—by creating a custom `StatefulWidget` called `AppStartupWidget`. This widget will encapsulate the app initialization logic and manage the different UI states.



A diagram showing the various widget trees loaded at different stages of the app startup process

Implementation

Here's a basic structure for `AppStartupWidget`:

```
class AppStartupWidget extends StatefulWidget {
  const AppStartupWidget({super.key});

  @override
  State<AppStartupWidget> createState() => _AppStartupWidgetState();
}

class _AppStartupWidgetState extends State<AppStartupWidget> {
  // declare state variables

  @override
  void initState() {
    unawaited(initializeDependencies());
    super.initState();
  }

  Future<void> initializeDependencies() {
    try {
      // async initialization
      await someAsyncCodeThatMayThrow();
      // TODO: set success state
    } catch (_) {
      // TODO: set error state
    }
  }

  @override
  Widget build(BuildContext context) {
    /*
     * if (success) return MainApp()
     * if (loading) return AppStartupLoadingWidget()
     * if (error) return AppStartupErrorWidget(error, onRetry: () { ...
    */
  }
}
```

With this widget in place, we can simplify the `main()` method to:

```
void main() {  
  runApp(const MaterialApp(home: AppStartupWidget()));  
}
```

Next Steps

To fully flesh out this widget, we could:

1. **Model the states:** Use a sealed class or an enumeration to represent the three states (`Loading`, `Error`, and `Success`).
2. **Handle async logic:** Populate `initializeDependencies()` with our async initialization code and update the state accordingly.
3. **Switch between states in the UI:** Use a `switch` expression in the `build()` method to render the appropriate widget for each state.
4. **Add retry functionality:** Provide a callback in the error state to retry initialization.

While this approach works well for stateful initialization, it's not sufficient if your app needs to initialize dependencies and make them easily accessible throughout the app. For that, we need to leverage a **dependency injection framework** or a **service locator**.

This is where **Riverpod** comes into play. Let's see how it can help us manage dependencies asynchronously. 🙌

Asynchronous Dependency Initialization with Riverpod

Previously, we used this simple initialization code:

```
await someAsyncCodeThatMayThrow();
```

But in a real-world app, you'll likely have dependencies that need to be ready for later use. Riverpod providers are perfect fit for this job. For example:

```
// A regular provider for accessing a dependency that is initialized  
*synchronously*  
@Riverpod(keepAlive: true)
```

```
FirebaseAuth firebaseAuth(FirebaseAuthRef ref)
=> FirebaseAuth.instance;
```

For dependencies that need **asynchronous** initialization, such as `SharedPreferences`, you can use a `FutureProvider`:

```
// A FutureProvider for accessing a dependency that is initialized
*asynchronously*
@Riverpod(keepAlive: true)
Future<SharedPreferences> sharedPreferences(SharedPreferencesRef ref)
=> SharedPreferences.getInstance(); // returns a Future
```

Here, we want the dependency to be initialized as soon as the app starts.

But keep in mind, Riverpod providers are **lazily initialized by default**—they are built when first used, **not** when declared. And the documentation says that if we want to **eagerly initialize** a provider, we can do so with a child widget.

Note how I've used `keepAlive: true` in the declarations above. This makes sense for dependencies that are initialized only once, during app startup.

Eager Provider Initialization with a Child Widget

Let's take it a step further and create a custom `appStartupProvider` that centralizes all asynchronous initialization:

```
@Riverpod(keepAlive: true)
Future<void> appStartup(Ref ref) async {
  ref.onDispose(() {
    // ensure we invalidate all the providers we depend on
    ref.invalidate(sharedPreferencesProvider);
  });
  // all asynchronous app initialization code should belong here:
  await ref.watch(sharedPreferencesProvider.future);
}
```

Key points:

1. **Dependency handling:** We eagerly initialize the `sharedPreferencesProvider` by awaiting its `.future`. This trick is [explained here](#).
2. **Cleanup:** The `onDispose` callback invalidates the `sharedPreferencesProvider` when the `appStartupProvider` is invalidated.

Now, let's update the `AppStartupWidget` to use this provider:

```
/// Widget class to manage asynchronous app initialization
class AppStartupWidget extends ConsumerWidget {
  const AppStartupWidget({super.key, required this.onLoaded});
  final WidgetBuilder onLoaded;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // 2. eagerly initialize appStartupProvider (and all the providers it
    depends on)
    final appStartupState = ref.watch(appStartupProvider);
    return appStartupState.when(
      // 3. loading state
      loading: () => const AppStartupLoadingWidget(),
      // 4. error state
      error: (e, st) => AppStartupErrorWidget(
        message: e.toString(),
        // 5. invalidate the appStartupProvider
        onRetry: () => ref.invalidate(appStartupProvider),
      ),
      // 6. success - now load the main app
      data: (_) => onLoaded(context),
    );
  }
}
```

Finally, simplify the `main()` method:

```
void main() {
  // 1. load it when the app starts
  runApp(const ProviderScope(
    child: MaterialApp(
      home: AppStartupWidget(
        onLoaded: () => MainApp(),
      ),
    ),
  ),
```



```
);  
}
```

How It Works

1. **Application Startup:** When the app starts, `AppStartupWidget` is loaded.
2. **Eager Initialization:** The `appStartupProvider` is eagerly initialized, along with all its dependencies (e.g., `sharedPreferencesProvider`).
3. **Loading State:** While the provider is in a `loading` state, we display the `AppStartupLoadingWidget`.
4. **Error Handling:** If initialization fails, the `AppStartupErrorWidget` is displayed with an option to retry.
5. **Retry Logic:** On retry, we invalidate the `appStartupProvider`, triggering re-initialization.
6. **Success State:** Once initialization succeeds, the `onLoaded` callback is triggered, and the `MainApp` widget takes the stage.

Accessing Eagerly-Initialized Providers with `requireValue`

Once `MainApp` is loaded, we can safely assume that the `sharedPreferencesProvider` is **fully initialized**. This allows us to access its value directly using `requireValue`:

```
@override  
Widget build(BuildContext context, WidgetRef ref) {  
  final sharedPrefs = ref.watch(sharedPreferencesProvider).requireValue;  
}
```

By using `requireValue`, you're essentially saying: *"I know this provider was initialized asynchronously, but at this point, it is guaranteed to have a value."*

This works seamlessly because the `sharedPreferencesProvider` (a `FutureProvider`) was **eagerly initialized** before `MainApp` was loaded. As a result, every descendant widget of `MainApp` can access it without additional checks.

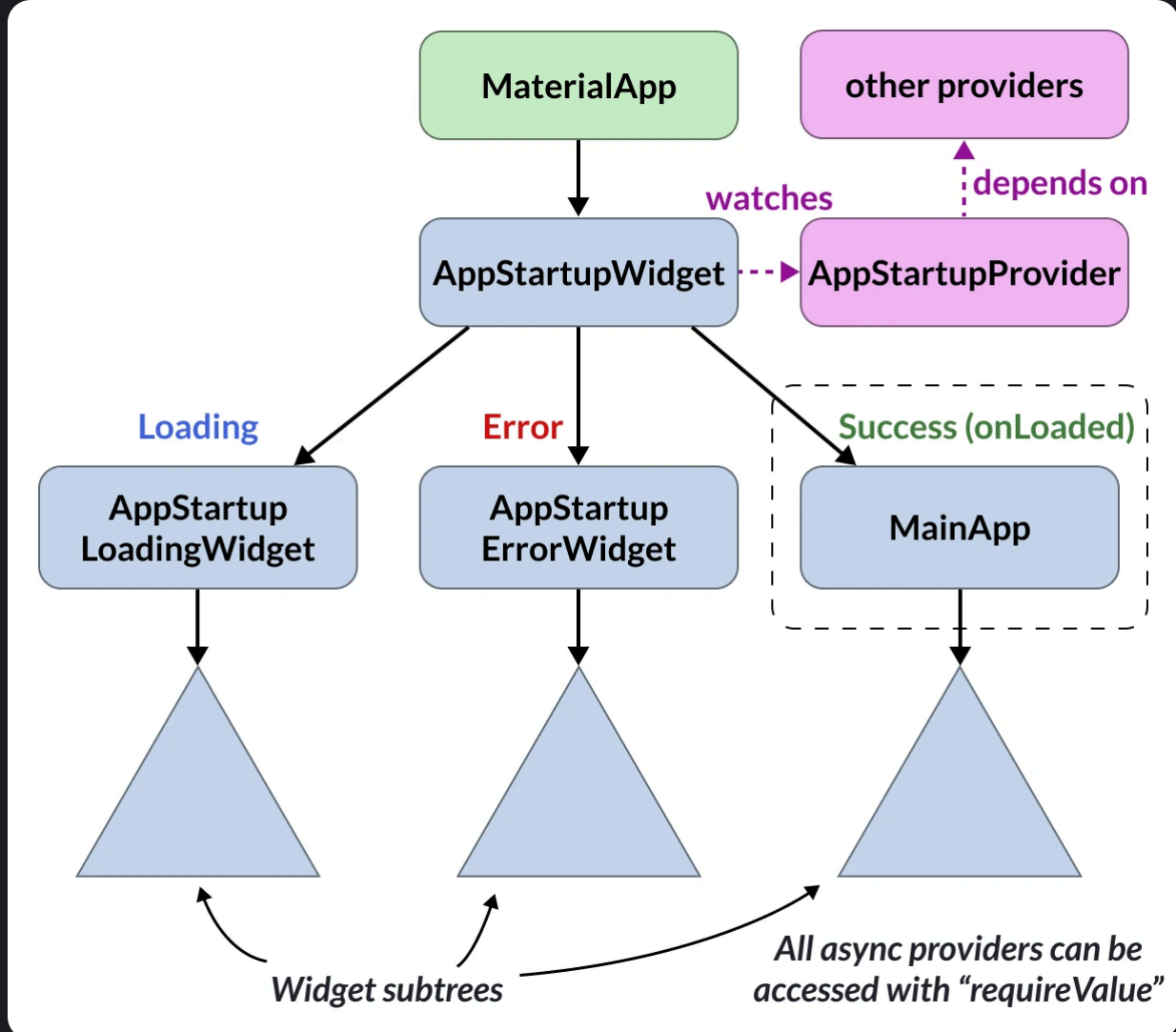


Diagram showing the main widgets and providers

Heads-up: If you try to access `requireValue` on a provider that isn't ready yet, you'll hit an exception. If that happens, it's time to debug and revisit your assumptions.

Important Note About URL Navigation and Deep Links

So far, we've seen how to eagerly initialize dependencies by placing an `AppStartupWidget` at the top of the widget tree. However, things get trickier when your app needs to support **URL navigation** or **deep links**.

Why? Because apps that rely on URL-based navigation typically use a `MaterialApp.router` with a `GoRouter` (or equivalent) for routing. Unfortunately, the `AppStartupWidget` doesn't handle this requirement—it only manages app initialization.

The Solution: Introducing a `RootAppWidget`

To support URL navigation and deep links, we can introduce a top-level `RootAppWidget` that configures the router while still handling app startup. Here's how:

```
void main() {
  runApp(const ProviderScope(
    // * Use RootAppWidget, not AppStartupWidget
    child: RootAppWidget(),
  ));
}

class RootAppWidget extends ConsumerWidget {
  const RootAppWidget({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final goRouter = ref.watch(goRouterProvider);
    return MaterialApp.router(
      routerConfig: goRouter,
      builder: (_, child) {
        return AppStartupWidget(
          onLoaded: (_) => child!,
        );
      },
      ...,
    );
  }
}
```

How It Works

With this setup, the `GoRouter` instance is initialized as soon as `runApp` is called. This ensures your app can **process deep links and URL navigation immediately**.

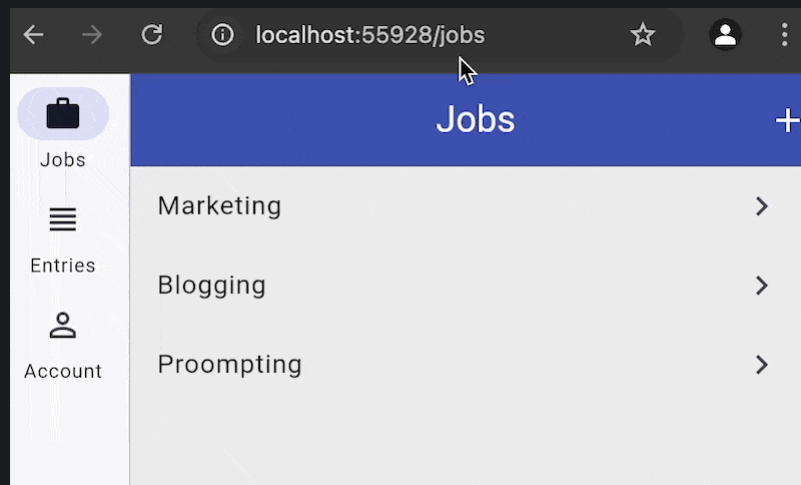
But there's one trick! Can you spot it?

The `MaterialApp.builder` is key to this setup. It allows us to wrap the router's `child` widget with the `AppStartupWidget` without interfering with the routing logic. This ensures that:

- The app startup logic runs before the main app UI loads.
- Deep links and URL-based navigation are processed correctly from the start.

Real-World Example

Here's a demo from my [Time Tracker app](#), where I test URL-based navigation with an artificial delay of one second during app startup:



App startup loading animation for my Time Tracker app

This setup works very well in practice, so let's summarize how everything ties together.

Summary: Stateful App Initialization with Router

Here's a diagram that captures the final setup:

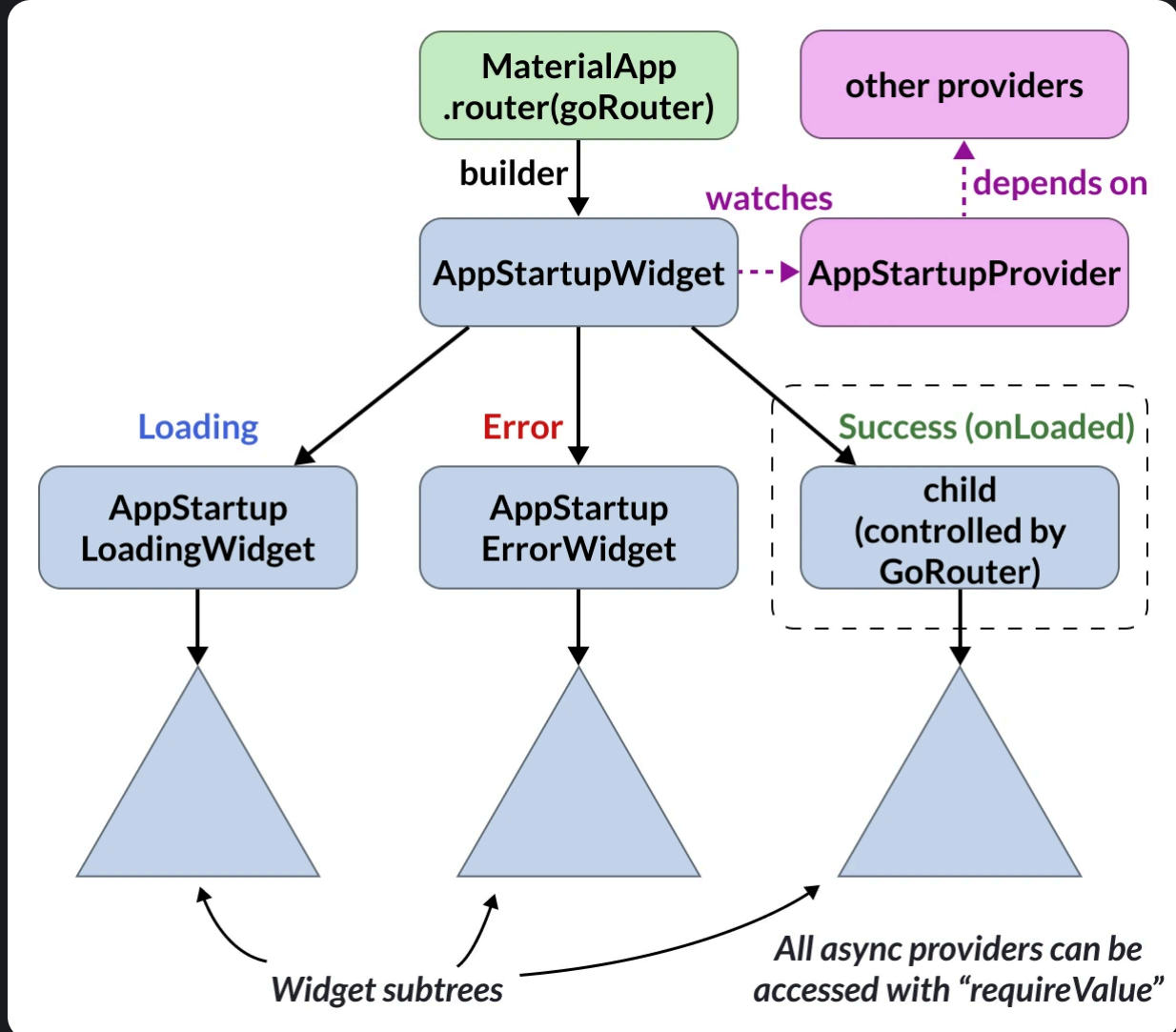


Diagram showing the main widgets and providers in combination with the `MaterialApp.router` API

To achieve a robust app initialization process with URL navigation and deep-link support, we need three key components:

1. `MaterialApp.router`: Configured with `GoRouter` (or an equivalent declarative routing solution) to handle URL navigation and deep links.
2. `AppStartupWidget`: Wrapped in `MaterialApp.builder` to manage the app's initialization flow while deferring to the router's child widget once initialization is complete.
3. `appStartupProvider`: A `FutureProvider` that eagerly initializes all asynchronous dependencies required during app startup.

By layering these components, we've created a production-ready solution that's flexible enough to handle both complex routing and robust dependency handling.

Common Questions

Now that we've covered the essentials, let's address some common questions you might have.

How to Eagerly Initialize Multiple Providers?

If your app requires multiple dependencies to be initialized upfront, the

`appStartupProvider` is the perfect place to manage them:

```
@Riverpod(keepAlive: true)
Future<void> appStartup(AppStartupRef ref) async {
  // all asynchronous app initialization code should belong here:
  await ref.watch(sharedPreferencesProvider.future);
  await ref.watch(sembastDatabaseProvider.future);
}
```

If the dependencies are **independent of each other**, you can even use `Future.wait` to initialize them concurrently. This can save precious milliseconds during startup:

```
@Riverpod(keepAlive: true)
Future<void> appStartup(AppStartupRef ref) async {
  // await for all initialization code to be complete before returning
  await Future.wait([
    ref.watch(sharedPreferencesProvider.future),
    ref.watch(onboardingRepositoryProvider.future)
  ]);
}
```

What About Programmer or Configuration Errors?

When it comes to dependency initialization, consider the type of failure you're handling:

- If initialization might fail due to **unexpected runtime errors** (e.g., network issues, missing permissions), it makes sense to handle it in `appStartup` so you can recover gracefully.
- If initialization can only fail because of a **programmer or configuration error**, it's better to perform it directly in `main`. Why? Because such errors are typically non-recoverable and should be caught and fixed during development.

A classic example is Firebase initialization, which is prone to configuration errors if not set up correctly:

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
```

```
// * Initialize Firebase
await Firebase.initializeApp(options:
DefaultFirebaseOptions.currentPlatform);
// * Entry point of the app
runApp(const ProviderScope(
  child: MainApp(),
));
}
```

By keeping this code in `main`, any configuration issues (e.g., missing or misconfigured `google-services.json` or `GoogleService-Info.plist`) will surface immediately during development, allowing you to quickly identify and fix them.

How to Implement the Retry Logic?

When dealing with complex initialization logic, it's often necessary to allow users to **retry** if something goes wrong. To achieve this, we can enhance the `appStartupProvider` by converting it into a `Notifier` with a `retry()` method:

```
@riverpod
class AppStartupNotifier extends _$AppStartupNotifier {
  @override
  Future<void> build() async {
    // Initially, load the database from JSON
    await _complexInitializationLogic();
  }

  Future<void> _complexInitializationLogic() async {
    // some complex initialization logic
  }

  Future<void> retry() async {
    // use AsyncValue.guard to handle errors gracefully
    state = await AsyncValue.guard(_complexInitializationLogic);
  }
}
```

Accordingly, the `AppStartupWidget` can be updated to call the `retry` method from the `onRetry` callback:

```

class AppStartupWidget extends ConsumerWidget {
  const AppStartupWidget({super.key, required this.onLoaded});
  final WidgetBuilder onLoaded;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // 1. eagerly initialize appStartupProvider (and all the providers it
    depends on)
    final appStartupState = ref.watch(appStartupNotifierProvider);
    return appStartupState.when(
      // 2. loading state
      loading: () => const AppStartupLoadingWidget(),
      // 3. error state
      error: (e, st) {
        return AppStartupErrorWidget(
          message:
            'Could not load or sync data. Check your Internet
            connection and retry or contact support if the issue persists.',
          // 4. retry logic
          onRetry: () async {
            await ref.read(appStartupNotifierProvider.notifier).retry();
          },
        );
      },
      // 5. success - now load the main app
      data: (_) => onLoaded(context),
    );
  }
}

```

Can Any Provider be Eagerly Initialized?

No, not all providers should be eagerly initialized.

The `appStartupProvider` is specifically designed for initializing **asynchronous dependencies** that remain constant throughout the app's lifecycle. Examples include shared preferences, database connections, or configuration files.

Providers that may change state over time (e.g., user authentication status or real-time data streams) should **not** be eagerly initialized. Doing so could lead to unwanted rebuilds or performance issues, as their state updates dynamically after the app has started.

How to Transition Between the Splash, Loading, and Main UI Screens?

By default, a Flutter app shows a **native splash screen** until `runApp` is called. You can configure this splash screen using a package like [flutter_native_splash](#).

To ensure a smooth transition between the splash screen, loading UI, and the main app UI:

1. **Match the Styles:** Customize your Flutter loading screen to visually match the native splash screen, creating the illusion of continuity.
2. **Add Transitions:** Overlay the loading screen with subtle animations or progress indicators to keep users engaged.
3. **Animate Into the Main UI:** Once initialization is complete, use a fade, slide, or scale animation to transition from the loading screen to the main app UI.

This approach ensures a polished, professional user experience while minimizing jarring transitions during app startup.

Can I Use Provider Overrides as a Simpler Alternative for Asynchronous Initialization?

The old way of doing things was to declare a normal provider that throws an

`UnimplementedError` by default:

```
@Riverpod(keepAlive: true)
SharedPreferences sharedPreferences(SharedPreferencesRef ref) =>
    throw UnimplementedError();
```

Then, in the `main` function, we'd override the provider with its initialized value:

```
void main() async {
    final sharedPreferences = await SharedPreferences.getInstance();
    runApp(ProviderScope(
        overrides: [
            sharedPreferencesProvider.overrideWithValue(sharedPreferences)
        ],
        child: const MainApp(),
    ));
}
```

This method effectively injects pre-initialized dependencies into the `ProviderScope` before the app starts.

When Should You Use This Approach?

If you're working with an **older codebase**, you might encounter this pattern—or variants that use `ProviderContainer` and `UncontrolledProviderScope` for eager initialization. While it works, it has limitations:

1. **No Retry Capability:** If the initialization fails (e.g., due to a runtime error), there's no built-in mechanism to retry or recover gracefully.
2. **Error Handling:** It assumes that initialization will always succeed, which might not be a safe assumption for all dependencies.

Should You Still Use It?

This approach is fine if you're **100% confident** that the initialization will always succeed. Dependencies like `SharedPreferences` or static configuration files are good candidates because they rarely fail in production.

In this article, I only used `SharedPreferences` as an example for illustration purposes. In practice, it's ok to asynchronously initialize `SharedPreferences` inside `main()`.

However, if there's any chance of failure (e.g., network-based initialization, database migrations), the stateful `AppStartupWidget` approach is a better choice. It provides more flexibility for error handling, retries, and user feedback during app startup.

Conclusion

A smooth and delightful onboarding experience is your chance to impress your users and set the tone for your app. To achieve this, your app startup logic needs to be **robust**, handle errors gracefully, and provide a seamless transition into the main app.

Here's a quick recap of the techniques we covered:

- Use an `appStartupProvider` to initialize all asynchronous dependencies, leveraging `await` and `.future`.

- Eagerly initialize the `appStartupProvider` inside a top-level `AppStartupWidget` and use `requireValue` to access asynchronously initialized dependencies once they are guaranteed to be ready.
- Provide a **loading UI**, handle errors gracefully, and include a retry mechanism for recoverable failures.
- For apps with **URL navigation or deep-linking**, integrate your `AppStartupWidget` within `MaterialApp.builder` to support declarative routing while managing startup logic.

That's it! You now have a repeatable process for writing robust app startup code.

Real-World Example

Looking for a practical implementation? Check out my [time-tracking app on GitHub](#), where I've applied these concepts to a real-world project.

What's Next?

While we've covered the essentials of app startup logic, there's still more to explore. For instance, we haven't discussed **error monitoring** or **crash reporting**, which are critical for identifying and resolving issues in production. I dive deeper into these topics (and more) in my latest course. 📌

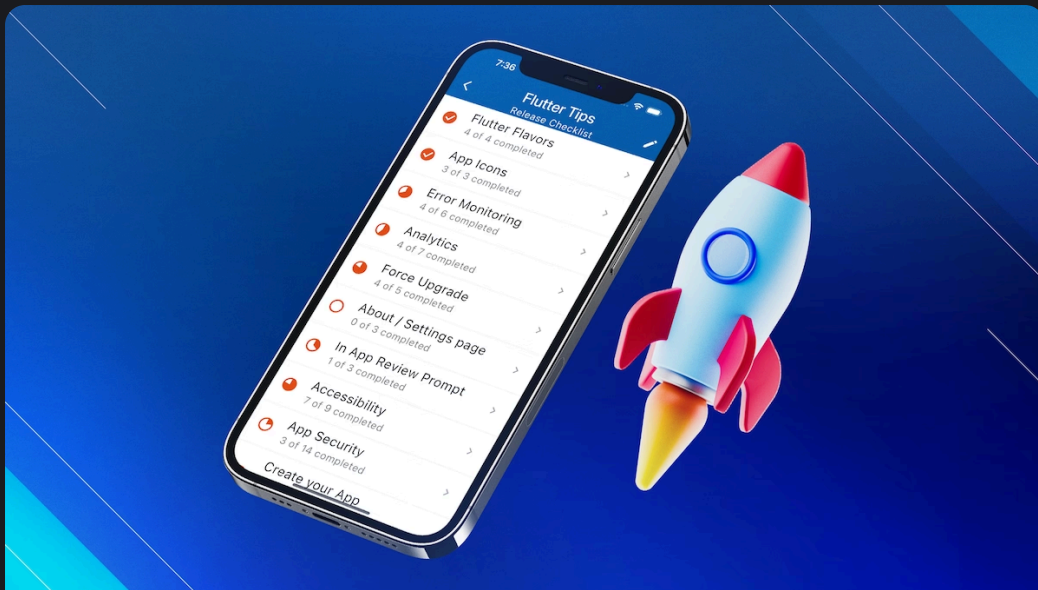
New Course: Flutter in Production

When it comes to **shipping** and **maintaining** apps in production, there are many important aspects to consider:

- **Preparing for release:** splash screens, flavors, environments, error reporting, analytics, force update, privacy, T&Cs
- **App Submissions:** app store metadata & screenshots, compliance, testing vs distribution tracks, dealing with rejections
- **Release automation:** CI workflows, environment variables, custom build steps, code signing, uploading to the stores
- **Post-release:** error monitoring, bug fixes, addressing user feedback, adding new features, over-the-air updates

My latest course will help you get your app to the stores faster and with fewer headaches.

If you're interested, you can learn more and enroll here. 📌



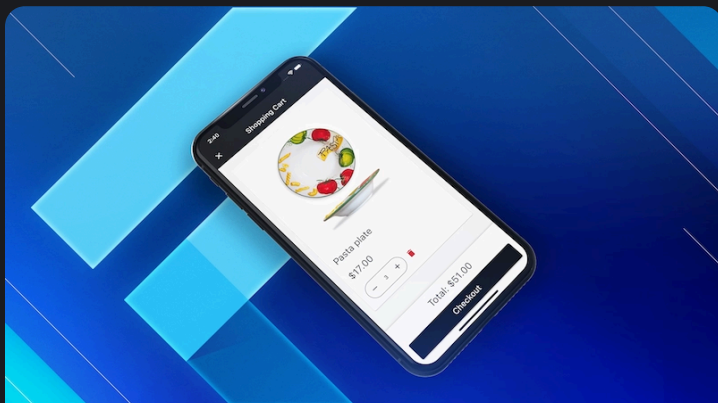
Flutter In Production

INTERMEDIATE TO ADVANCED

Learn about flavors, environments, error monitoring, analytics, release management, CI/CD, and finally ship your Flutter apps to the stores. 🚀

Want More?

Invest in yourself with my high-quality Flutter courses.



Flutter Foundations Course

INTERMEDIATE TO ADVANCED

Learn about State Management, App Architecture, Navigation, Testing, and much



Flutter & Firebase Masterclass

INTERMEDIATE TO ADVANCED

19

more by building a Flutter eCommerce app on iOS, Android, and web.

Learn about Firebase Auth, Cloud Firestore, Cloud Functions, Stripe payments, and much more by building a full-stack eCommerce app



The Complete Dart Developer Guide

BEGINNER

Learn Dart Programming in depth. Includes: basic to advanced topics, exercises, and projects. Last updated to Dart 2.15.



Flutter Animations Masterclass

INTERMEDIATE

Master Flutter animations and build a completely custom habit tracking application.



CODE WITH ANDREA

Copyright © 2020-present Coding With Flutter Limited

[Contact](#)

[Twitter](#)

[Discord](#)

[GitHub](#)

[RSS](#)

[Meta](#)

[Privacy Policy](#)

[Terms of Use](#)